

Self-Consolidating Language Models: Continual Knowledge Incorporation from Context

Zekun Wang* Anant Gupta* Zihan Dong Christopher J. MacLellan
Georgia Institute of Technology
{zekun, agupta886, zdong312, cmaclell}@gatech.edu

Abstract

Large language models (LLMs) increasingly receive information as streams of passages, conversations, and long-context workflows. While longer context windows expose more evidence, they do not ensure that useful information is preserved and reused. We study continual context consolidation: writing current context into model weights while limiting interference with previously consolidated information. We propose **Self-Consolidating Language Models (SCoL)**, a post-training framework in which, given current context, an LLM learns to generate textual update instructions specifying which of its own Transformer layers should be updated. Because committed updates change the model that later generates future selections, we train SCoL with meta-reinforcement learning over an evolving model state. We instantiate SCoL with supervised QA rewards on SQuAD knowledge incorporation and intrinsic likelihood-based rewards for LongBench v2 long-context consolidation. Across both settings, SCoL improves acquisition and retention over prompting, summarization, batch test-time training, and sequential finetuning baselines. Analysis of learned selection patterns shows that SCoL encourages the LLM to generate sparse update locations that align with layers of high Fisher information, suggesting that the model learns to route plasticity toward loss-sensitive regions while limiting interference. Moreover, SCoL transfers from shorter meta-training streams to longer LongBench v2 streams at evaluation, suggesting that our framework supports scalable streaming consolidation.

1 Introduction

Large language models increasingly operate over streaming context, including multi-turn conversations, tool-using agents, document workflows, and long-horizon interactive tasks. In these settings, the model must not only answer from the current prompt, but also preserve earlier information for later use. Longer context windows help, but they do not guarantee reliable memory: models can underuse distant evidence and degrade as context length and reasoning complexity increase [1, 2]. Prior work manages long-context information through retrieval or external memory [3, 4, 5], attention-routing or memory mechanisms [6], compression [7, 8], and agentic memory stores [9, 10]. These methods are effective, but they often depend on retrieval quality, auxiliary modules, lossy compression, or repeated inference-time processing of persistent context. We study a complementary direction: using the model weights as an internal memory substrate. Motivated by theories of memory consolidation, where experience is integrated into long-term internal representations rather than kept only as transient context [11, 12, 13], we ask whether useful context can be written into parameters so it remains available after the original context is gone.

This reframes long context inference as a continual learning problem. The model must consolidate a stream of contexts into weights while preserving previously consolidated knowledge. However,

*Equal contribution.

sequential neural network updates are prone to catastrophic interference [14]. Classical methods reduce interference by estimating parameter importance, for example with Fisher based regularization [15], but reliable Fisher or Hessian style estimation is costly and implementation sensitive at LLM scale [16]. We therefore ask whether the model itself can determine where adaptation should occur.

Inspired by SEAL [17], we propose *Self-Consolidating Language Models* (SCoL), a post-training framework in which a LLM learns to generate textual update instructions specifying with its own parameters to adapt for a given context. We then attach LoRA adapters to the selected modules and perform a LoRA update [18]. Unlike SEAL, which focuses on generating self edits and update directives, our focus is where consolidation should occur under a drifting policy—each committed update changes the model that will choose future updates, so the policy itself is part of the evolving state. We trained SCoL with meta-reinforcement learning whose reward favors acquisition of the current context while penalizing forgetting of earlier contexts.

We instantiate SCoL under two reward settings: a supervised QA reward for SQuAD-style knowledge incorporation [19], and an intrinsic likelihood-based reward for unlabeled long-context streams evaluated on LongBench v2 [20]. Across both settings, SCoL improves acquisition and retention over in-context inference, summarization, batch test-time training, and sequential fine-tuning. Its learned selections are sparse and align with high-Fisher information layers, suggesting that SCoL routes plasticity toward loss-sensitive regions while limiting interference. On LongBench v2, selections meta-trained on shorter streams also transfer to longer evaluation streams without full-context access.

Our contributions are:

- We recast long-context inference as continual context consolidation, where useful information from a stream of contexts is written into model weights while preserving previously consolidated knowledge.
- We proposed SCoL, a post-training framework that trains an LLM with meta-reinforcement learning to generate textual description of where in its own weights to update, and instantiate SCoL with supervised QA rewards and intrinsic likelihood-based rewards, covering both labeled knowledge incorporation and unlabeled long-context consolidation.
- We evaluate against in-context inference, summarization, batch test-time training, and sequential fine-tuning baselines on SQuAD and LongBench v2, and show that teaching the LLM to generate update-location selections improves acquisition and retention, produces sparse selections that align with Fisher information, and transfers from shorter training streams to longer LongBench v2 context streams.

2 Related work

Continual learning for large language models Continual learning studies how models acquire new tasks, domains, or knowledge without overwriting prior behavior. In LLMs, this appears in factual and lifelong model editing [21, 22, 23, 24], continual domain adaptation [25], and continual instruction or alignment tuning [26, 27]. Existing methods typically control forgetting by estimating parameter importance or preserving prior behavior through regularization [15, 28], or by restricting updates to low-rank, orthogonal, or modular parameter subspaces [18, 29, 30]. These approaches are effective but often depend on fixed update rules, replay or distillation signals, hand-designed parameter partitions, or explicit importance estimates that can be costly at LLM scale [16]. Our work instead learns a context-conditioned update policy: rather than computing weight importance directly, we post-trained the LLM to decide where new contextual knowledge should be inserted so adaptation improves incorporation while minimizing forgetting.

Inference with long context Long-context inference asks whether an LLM can reliably use information that appears far back in the prompt. Although context windows continue to grow, models can still underuse distant evidence and degrade as contexts become longer or more complex [1, 2]. Existing approaches improve access to long-range information through retrieval or external memory [3, 31, 4, 5], special attention or memory mechanisms [6, 5], context compression [32, 7, 8], or agentic memory systems [10, 9, 33, 34]. These methods are effective but often keep information outside the model weights or compress it before reuse, making performance depend on retrieval quality, memory management, or lossy summarization. We study a complementary direction: consolidating context-derived knowledge into localized parameter updates. This reframes long-context inference as

continual knowledge incorporation, where the model must retain useful information from a stream of contexts without overwriting previously consolidated knowledge.

Self-adaptation with reinforcement learning Reinforcement learning is often used to optimize LLMs as policies over text for alignment, reasoning, or task-level rewards [35, 36]. In self-adapting LMs, the target instead shifts toward learning *how the model should change* in response to new information, connecting to meta-learning and expert-iteration views of controlling an inner improvement process [37]. Recent work learns such adaptation strategies: Transformer-Squared learns task-conditioned self-adaptation with RL [38], and SEAL trains an LLM to generate self-edits and update directives [17]. CaMeLS is especially close to our setting because it meta-learns online adaptation over context, but it does so by training a separate small autoregressive model to assign token-level loss weights during fine-tuning [39]. By contrast, we proposed a post-training paradigm in which the LLM itself learns to propose *where in its own weights* to update, without explicitly computing parameter importance. Since each committed update changes the model used for future contexts, we frame the problem as a meta-RL problem over a drifting model state, with the goal of continual knowledge consolidation under forgetting constraints.

3 Self-Consolidating Language Models

We study a continual consolidation setting in which a single LLM ingests a stream of contexts and must internalize each into its weights, without replay and without retrieval at inference time. Section 3.1 formalizes continual context consolidation as a reinforcement learning problem whose reward decomposes into an acquisition term and a forgetting term. Section 3.2 presents SCoL, a meta-reinforcement learning procedure that trains the LLM to generate textual update-location selections under a drifting model state. Section 3.3 instantiates the reward under supervised and sparse-supervision regimes. The full procedure is given in Algorithm 1.

3.1 Problem setup: continual context consolidation

Let π_{θ_0} denote a base language model (our policy) with parameters $\theta_0 \in \mathbb{R}^d$, and let $\mathcal{C} = (c_1, c_2, \dots, c_T)$ be a stream of contexts, where each $c_t \in \mathcal{X}^*$ is a token sequence. We use *context* generically: it can be a text passage, a multi-turn interaction history, or a long-context window. At step t , the current model $\pi_{\theta_{t-1}}$ receives c_t and generates a *textual* action

$$a_t \sim \pi_{\theta_{t-1}}(\cdot | c_t), \quad (1)$$

which is parsed into a structural update units, and in our main instantiation the units are Transformer layers. We then update the current model by:

$$\Delta\theta_t = \text{Adapt}(\theta_{t-1}, c_t, a_t), \quad \theta_t = \theta_{t-1} \oplus \Delta\theta_t. \quad (2)$$

The prompting template and parsing procedure are described in Section A. The weights θ_t carry all accumulated history. No retrieval index or auxiliary memory is maintained for generation. During training, we keep a past-context set $\mathcal{D}_{<t}$ containing only the material needed to *evaluate* forgetting from steps $1, \dots, t-1$. At inference time, the model relies on θ_t alone. After each consolidation step, the environment returns

$$r_t(\theta_t; c_t, \mathcal{D}_{<t}) = u(\theta_t; c_t) - \lambda f(\theta_t; \mathcal{D}_{<t}), \quad (3)$$

where u measures acquisition of the current context, f measures drift on previously consolidated material, and $\lambda \geq 0$ trades off acquisition and retention. We keep u and f abstract here and instantiate them in Section 3.3.

The learning objective is to learn a *pre-adaptation* policy π_{θ_0} whose own sequential updates produce high cumulative reward:

$$\max_{\theta_0} \mathcal{J}(\theta_0) = \mathbb{E}_{\substack{a_t \sim \pi_{\theta_{t-1}}(\cdot | c_t) \\ \theta_t = \theta_{t-1} \oplus \text{Adapt}(\theta_{t-1}, c_t, a_t)}} \left[\sum_{t=1}^T r_t(\theta_t; c_t, \mathcal{D}_{<t}) \right]. \quad (4)$$

The key point is that the policy parameters drift during the stream: the action at step t is sampled from $\pi_{\theta_{t-1}}$, whose parameters already contain all committed updates from earlier contexts. Thus,

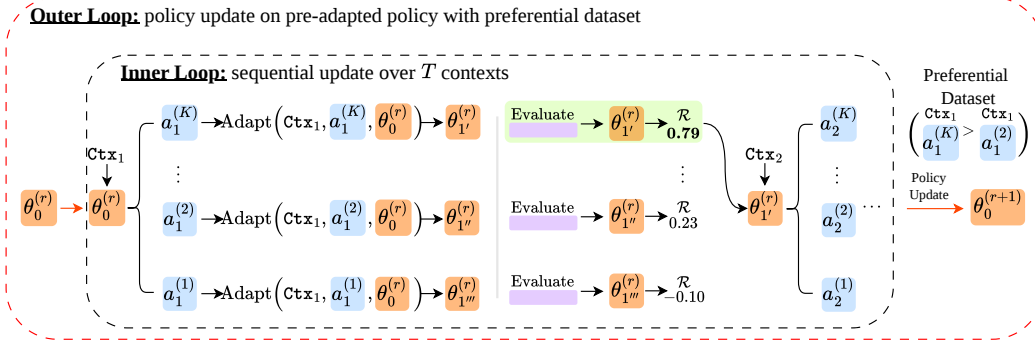


Figure 1: Training pipeline of the Self-Consolidating Language Model (SCoL). **Inner loop (black):** at each context Ctx_t , the current model state $\theta_{t-1}^{(r)}$ samples K textual actions, adapts a candidate model, and scores it with the reward in Equation (3). The highest-reward candidate is committed, advancing the running model state. **Outer loop (red):** after the stream is exhausted, the induced preferential dataset trains the round-start policy $\theta_0^{(r)}$ with IPO, producing $\theta_0^{(r+1)}$ for the next round.

the goal is not to learn a fixed context-to-update mapping, but to meta-learn an initialization whose rolled-out updates continue to produce good future update selections.

Several nearby paradigms share aspects of this problem but differ in a central architectural commitment. RAG [3] and memory-augmented architectures [9, 7] handle new information by expanding an external store while leaving model parameters fixed. Knowledge-editing methods [21, 40, 22, 41] target isolated factual updates rather than streaming accumulation and general context learning. Batch fine-tuning [42, 43] requires simultaneous access to the full $\mathcal{C}$. SCoL instead commits each incoming context into the model weights online, while learning where such updates should occur.

3.2 Learning to learn without forgetting with meta-reinforcement learning

SCoL parameterizes the adaptation decision as text generation. For each context, the LLM emits a textual list of update locations. We then attach LoRA modules [18] to the selected structural units and perform a CLM update on c_t :

$$\Delta\theta^*(a_t) = \arg \min_{\Delta\theta_{a_t}} \mathbb{E}_{x \sim c_t} [-\log p_{\theta_{t-1} + \Delta\theta_{a_t}}(x)]. \quad (5)$$

This yields the candidate post consolidation model

$$\theta_t^{(a)} = \theta_{t-1} \oplus \Delta\theta^*(a). \quad (6)$$

Across candidates, the inner-loop hyperparameters are fixed; candidates differ only in which structural units receive adapters. Thus, the learned decision is where to adapt, not how strongly to adapt. At each context, we sample K candidate textual actions from the current, already-drifted policy:

$$a_t^{(1)}, \dots, a_t^{(K)} \sim \pi_{\theta_{t-1}}(\cdot | c_t). \quad (7)$$

Each action is parsed, adapted, and scored:

$$r_t^{(k)} = r_t(\theta_t^{(a_t^{(k)})}; c_t, \mathcal{D}_{<t}). \quad (8)$$

We then commit the highest-reward candidate,

$$k^* = \arg \max_k r_t^{(k)}, \quad \theta_t \leftarrow \theta_t^{(a_t^{(k^*)})}. \quad (9)$$

Algorithm 1: Self-Consolidating LM Training: Learning to Learn Without Forgetting

Require: $\theta_0^{(1)}$, stream $\mathcal{C} = (c_1, \dots, c_T)$, K, λ , rounds R
Ensure: trained policy $\pi_{\theta_0^{(R+1)}}$

```

1: for  $r = 1, \dots, R$  do
2:    $\theta_0^{(r)} \leftarrow \theta_0^{(r)}$ ;  $\mathcal{P}^{(r)} \leftarrow \emptyset$ 
3:   for  $t = 1, \dots, T$  do
4:      $a_t^{(1)}, \dots, a_t^{(K)} \sim \pi_{\theta_{t-1}^{(r)}}(\cdot | c_t)$ 
5:     for  $k = 1, \dots, K$  do
6:        $\Delta_t^{(k)} \leftarrow \text{ADAPT}(\theta_{t-1}^{(r)}, c_t, a_t^{(k)})$ 
7:        $\theta_t^{(r,k)} \leftarrow \theta_{t-1}^{(r)} \oplus \Delta_t^{(k)}$ 
8:        $r_t^{(k)} \leftarrow u(\theta_t^{(r,k)}; c_t) - \lambda f(\tilde{\theta}_t^{(r,k)}; \mathcal{D}_{<t})$ 
9:     end for
10:    append  $\{(a^w, a^l, c_t) : r_w > r_l\}$  to  $\mathcal{P}$ 
11:     $k^* \leftarrow \arg \max_k r_t^{(k)}$ 
12:     $\theta_t^{(r)} \leftarrow \tilde{\theta}_t^{(r,k^*)}$ 
13:   end for
14:    $\theta_0^{(r+1)} \leftarrow \arg \min_{\theta} \mathcal{L}_{\text{IPO}}(\theta; \mathcal{P}^{(r)})$ 
15: end for
16: return  $\pi_{\theta_0^{(R+1)}}$ 

```

This committed update changes the model that will generate the next action: π_{θ_t} is both the consolidated model and the policy used at the next context. The K candidate scores also induce preference data. For each context, we construct

$$\mathcal{P}_t = \{(c_t, a^w, a^l) : r_t(a^w) > r_t(a^l)\}, \quad (10)$$

where a^w and a^l denote higher- and lower-reward textual actions for the same context. Across the full inner stream, we collect $\mathcal{P}^{(r)} = \bigcup_{t=1}^T \mathcal{P}_t$.

After a full round of T sequential consolidations, we update the *pre-adaptation* parameters $\theta_0^{(r)}$, not the final drifted parameters $\theta_T^{(r)}$. This is the meta-learning step: the preference data are generated by rolling the current policy through a stream of self-induced updates, but the outer update improves the round-start model so that its future rollouts generate better update-location selections. We use Identity Preference Optimisation (IPO) [44]:

$$\mathcal{L}_{\text{IPO}}(\theta) = \mathbb{E}_{(c, a^w, a^l) \sim \mathcal{P}^{(r)}} \left[\left(\log \frac{\pi_{\theta}(a^w | c)}{\pi_{\text{ref}}(a^w | c)} - \log \frac{\pi_{\theta}(a^l | c)}{\pi_{\text{ref}}(a^l | c)} - \frac{1}{2\beta} \right)^2 \right], \quad (11)$$

with $\pi_{\text{ref}} = \pi_{\theta_0^{(r)}}$ fixed to the round-start policy. The minimizer becomes the next round-start policy,

$$\theta_0^{(r+1)} = \arg \min_{\theta} \mathcal{L}_{\text{IPO}}(\theta). \quad (12)$$

The next round begins from $\theta_0^{(r+1)}$ and produces a fresh drifting trajectory $\theta_1^{(r+1)}, \dots, \theta_T^{(r+1)}$. We use IPO rather than supervised fine-tuning on the top-1 action because the action space consists of low-entropy structured strings. In preliminary experiments, SFT-style reinforcement such as ReST [45] quickly concentrated probability mass on a few repeated layer lists. IPO instead stops updating once the chosen–rejected log-ratio reaches the finite margin $1/(2\beta)$, which helps preserve diversity among plausible update-location selections. Additional comparisons between outer-loop update algorithms are provided in Section A.11.

3.3 Reward instantiations and reward sparsity

The abstract reward of Equation (3) admits two instantiations, distinguished by whether a context arrives with downstream supervision.

When c_t is accompanied by a set of held-out queries $\mathcal{Q}_t = \{(q, a)\}$ tied to the context, both reward terms are measured in the same units of downstream performance. The acquisition term is the accuracy on $\mathcal{Q}_t$ of the candidate post-consolidation model. The forgetting term is the accumulated degradation on past query sets, measured against each past context’s own first-consolidation baseline. Concretely, for each past step $s < t$ we cache the accuracy $b_s = \text{Acc}_{\mathcal{Q}_s}(\theta_{s+1})$ attained immediately after c_s was first consolidated, and we penalise any drop from this baseline under the candidate state $\theta_t^{(a)}$. That is, $\mathcal{D}_{<t} = \{(c_s, \mathcal{Q}_s, b_s)\}_{s<t}$ in this regime, and

$$r(a; c_t, \theta_t) = \text{Acc}_{\mathcal{Q}_t}(\theta_t^{(a)}) - \frac{\lambda}{t-1} \sum_{s=1}^{t-1} [b_s - \text{Acc}_{\mathcal{Q}_s}(\theta_t^{(a)})]. \quad (13)$$

Many settings of interest do not provide such supervision. Contexts stream in without labels; downstream performance is unavailable, noisy, or delayed. In this regime $\mathcal{D}_{<t} = \{c_s\}_{s<t}$, and we instantiate both reward terms intrinsically. A well-consolidated model θ' should raise the likelihood of c_t relative to the pre-adaptation model θ_t ,

$$u_{\text{intrinsic}}(\theta'; c_t) = \log p_{\theta'}(c_t) - \log p_{\theta_t}(c_t), \quad (14)$$

and maintain the likelihood of $x \in \mathcal{D}_{<t}$ as measured by the output-distribution drift on previously-consolidated material. Because $\mathcal{D}_{<t}$ is an empirical collection of past contexts rather than samples drawn from the model, absolute log-likelihood differences can disproportionately weight passages with higher baseline likelihood or lower entropy, leading to uneven retention where some contexts dominate the forgetting signal. To ensure uniform retention across previously consolidated material, we instead measure forgetting in relative terms by normalizing the change in log-likelihood by its pre-adaptation value. This yields a notion of fractional degradation that treats each passage

comparably, regardless of scale, and better aligns with the objective of preserving all contexts without bias. Concretely, we define

$$\tilde{f}_{\text{intrinsic}}(\theta'; \mathcal{D}_{<t}) = \mathbb{E}_{x \sim \mathcal{D}_{<t}} \left[\frac{\log p_{\theta_t}(x) - \log p_{\theta'}(x)}{|\log p_{\theta_t}(x)|} \right] = \mathbb{E}_{x \sim \mathcal{D}_{<t}} \left[\frac{u_{\text{intrinsic}}(\theta'; x)}{\log p_{\theta_t}(x)} \right], \quad (15)$$

While this normalization ensures that each passage contributes comparably, it removes the original scale of log-likelihood differences. To restore a meaningful magnitude to the forgetting signal, we rescale by the average pre-adaptation log-likelihood over $\mathcal{D}_{<t}$, yielding

$$f_{\text{intrinsic}}(\theta'; \mathcal{D}_{<t}) = \tilde{f}_{\text{intrinsic}}(\theta'; \mathcal{D}_{<t}) \cdot \mathbb{E}_{x \sim \mathcal{D}_{<t}} [|\log p_{\theta_t}(x)|], \quad (16)$$

In both regimes the forgetting term is a drop relative to a per-context baseline evaluated on the same test material, differing only in whether that material is labeled queries or the raw context. Substituting Equations (14) and (16) into Equation (3) yields

$$r_{\text{sparse}}(a; c_t, \theta_t) = u_{\text{intrinsic}}(\theta_t^{(a)}; c_t) - \lambda f_{\text{intrinsic}}(\theta_t^{(a)}; \mathcal{D}_{<t}), \quad (17)$$

applicable to rolling-window long-context consolidation [46, 7], memory-compaction in agentic interaction histories [9, 47], and any setting in which a context arrives unlabeled.

We then evaluate our self-consolidating LMs framework under the above two reward instantiations: context that comes with immediate downstream supervisions (Section 4), and context that has no downstream supervisions (Section 5).

4 Continual knowledge injection

4.1 Experiment setup

Task and dataset. We evaluate continual knowledge incorporation, in which a language model is updated in sequence on a stream of SQuAD [19] passages and held responsible for answering a short question-answer set attached to each passage. We prepare a disjoint training split used for meta-training the policy and a validation split reserved for sequential-update evaluation. Additional dataset details are provided in Section A.7.

Policy, inner adaptation, and meta update. We choose the policy as a Qwen2.5-7B-Instruct model: at each context, the model is prompted to emit a list of up to $B = 10$ transformer-layer indices. Additional results for finer granularities (i.e., projection layers) are discussed in Section A.12. Our inner adaptation follows SEAL setting: for every candidate action, we attach a LoRA adapter to the selected layers and run the CLM update of Equation (5) on a set of implications pre-generated once per passage by a frozen SEAL-pretrained model, for 10 epochs at learning rate 2×10^{-4} and batch size 1. During meta-training we stream $T = 50$ passages per round; at each passage we sample $K = 10$ candidates actions from the current policy, score each candidate under of Equation (13) using a frozen Qwen2.5-7B-Instruct as a judge, commit the argmax to the running weights per Equation (9), and accumulate pairwise comparisons of reward scores into a round-level buffer. At the end of each round the buffer is used to reinforce the *pre-adaptation* base policy with IPO (Equation (11)), producing the base policy that seeds the next round; we report $R = 2$ such rounds. During evaluation, LLM only emits the selection once (i.e. $K = 1$) and commits the update. Prompts and additional training details are in Sections A.1 to A.5.

Baselines and metrics. We compare against three baselines and three SCoL variants. All methods use the same pre-generated implications and inner LoRA update procedure, so the comparison isolates the effect of *where* adaptation is applied. **QA Prompting** prompts the base model with the question only. **Batch TTT** trains a LoRA adapter jointly on all evaluation passages in an offline pass, serving

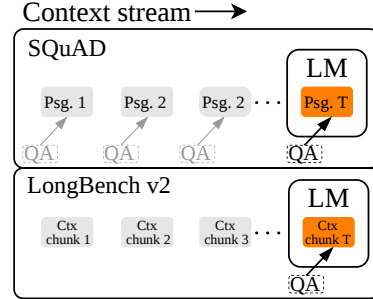


Figure 2: Illustrations of experiment setup under two reward settings. **Top:** each context (SQuAD passages) comes with an environment reward (downstream QA). **Bottom:** QA is evaluated at the end of the context stream.

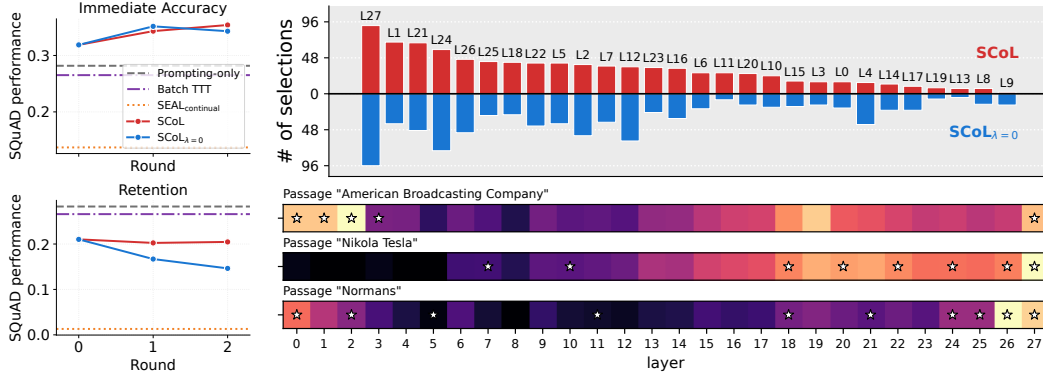


Figure 3: **Left:** immediate accuracy (top) and retention (bottom) across IPO updates. **Top Right:** top layer selection frequencies after the final IPO update, with the full reward variant in red and the acquisition only variant in blue. **Bottom Right:** examples of per-layer weight importance measured by Fisher information (brighter is higher) with SCoL’s selected layers marked by white stars.

as a batch-access upper bound. **SEAL**_{continual} applies the SEAL knowledge-incorporation procedure to the continual stream and serves as the continual LoRA fine-tuning baseline. **SCoL** _{$\lambda=0$} removes the forgetting term from our reward, isolating its contribution. **SCoL** uses the full reward with $\lambda = 1$.

At evaluation we apply each method to a fixed validation stream of $N = 100$ passages and record the sequential-update accuracy matrix $M \in [0, 1]^{N \times N}$ whose entry $M_{i,j}$ is the accuracy on the j -th passage’s queries after the model has been edited through the i -th passage (for $j \leq i$). From M we report two metrics for each round: **immediate acquisition**, the mean of the diagonal, measuring how well each update works at the moment it is committed and **retention**, the mean of the final-row entries on past passages, measuring accuracy on everything learned earlier once the full stream has been processed. Metric formulas and baseline details are given in Sections A.6 and A.8.

4.2 Results and discussions

Table 1 shows that both SCoL and SCoL _{$\lambda=0$} exceed all three baselines in immediate accuracy, indicating that the learned policy improves acquisition beyond prompting, batch TTT, and sequential updating. Batch TTT performs worse than prompting only, despite seeing all 100 passages and implications jointly, with an immediate accuracy drop from 28.17% to 26.52%. We interpret this as a declarative training, query evaluation mismatch. The adapter is trained on passage text and implication strings under language modeling, but is later queried through QA. This resembles the reversal curse [48], where facts learned in one direction are not reliably retrieved in another. SEAL collapses under sequential updating, consistent with compounding drift in parameter efficient continual tuning [49, 29]. Retention clarifies the role of the forgetting term. Prompting only and Batch TTT are no update reference points whose retention equals to their immediate score. SEAL retains only 1.30% accuracy after sequential incorporation. In contrast, SCoL _{$\lambda=0$} achieves stronger immediate acquisition, but its retention remains limited at 14.64%. Using forgetting reward f recovers retention to 20.46% while also improving immediate accuracy. This suggests that explicitly rewarding retention can substantially reduce catastrophic forgetting in a self-improving language model framework [17]. Figure 3 further shows that retention decays monotonically without f , whereas SCoL keeps retention nearly stationary and continues improving acquisition. The layer selection patterns in Figure 3 indicate that the learned policy is not selecting arbitrary update sites. Both SCoL and SCoL _{$\lambda=0$} concentrate their selections in the last layer (L27). This agrees with prior evidence that late blocks in autoregressive Transformers act as storage and retrieval sites for factual associations [50, 21]. We further test

Table 1: Comparison of immediate accuracy and retention across methods on continual knowledge incorporation over 100 SQuAD passages.

Method	Immediate Acc.	Retention
Prompting-only	28.17%	–
Batch TTT	26.52%	–
SEAL _{continual}	13.70%	1.30%
SCoL _{$\lambda=0$}	34.30%	14.64%
SCoL	35.40%	20.46%

a fixed-selection ablation that always updates the last 10 layers. It achieves 28.78% immediate accuracy and 20.10% retention, close to but below SCoL’s 20.46%. This supports the view that late layers preserve factual associations, and that SCoL learns to exploit this structure, while its stronger immediate accuracy suggests that context-dependent layer selection is more robust than a fixed late-layer strategy. At the same time, both variants avoid several early and middle layers, with $L8$ and $L9$ rarely selected and changing little across IPO rounds (see Figure 7). We refer to Appendix A.9 for additional results and standard errors.

Fisher alignment. We then ask what the LLM is learning to select, and how these generated layer selections contribute to retention. Since Fisher information measures where the current passage loss is most sensitive in the running model, it provides a direct diagnostic for whether SCoL routes adaptation toward steep layerwise directions while keeping updates sparse. We measure layerwise Fisher information on the running model’s base projection weights from each passage’s implication lines, with details in Appendix A.14. Given the layer selection set $S(c)$ generated for context c , we measure alignment as $\text{recall}@k(c) = |S(c) \cap \text{Top}_{k(c)}(F(c))|/k(c)$, where $k(c) = |S(c)|$ and $F(c)$ is the layerwise Fisher vector. A random selection of 10 layers (out of 28 total layers) has expected recall $10/28 = 35.7\%$.

Table 2 shows that SCoL reaches $45.1\% \pm 1.6\%$, while $\text{SCoL}_{\lambda=0}$ reaches $41.0\% \pm 1.3\%$, both clearly above the random selection baseline. This provides evidence that the learned selections are not merely produced by the prompt format and do not degenerate to random sparse choices, and shows that IPO reinforces the LLM to generate update locations that follow high sensitivity layers for the current passage. Figure 3 shows SCoL selections overlaid on Fisher information heatmaps for sampled SQuAD passages, where the generated layer selections concentrate on high (bright) Fisher layers. As a result, SCoL updates layers that are most responsive to the current passage, but does so under a sparse selection pattern that leaves most layers untouched. Such selective plasticity matches the principle behind continual learning methods that reduce catastrophic interference by restricting adaptation to sparse, task specific update pathways [51, 52, 53].

Table 2: Layerwise Fisher alignment on 100 SQuAD passages. Values report $\text{recall}@k(c)$ against the Fisher top- $k(c)$ layers for learned and random selections.

Method	$\text{recall}@k(c)$	
	Learned selection	Random selection
SCoL	45.1 ± 1.6	35.7
$\text{SCoL}_{\lambda=0}$	41.0 ± 1.3	35.7

5 Long-context consolidation

5.1 Experiment setup

Task, dataset, and evaluation. In many real world settings, models often receive information as a continuous stream (e.g., daily updates to knowledge base) without intermediate supervision, and must answer queries based on accumulated context. To simulate this setting, we use LongBench-v2 [20], where each example consists of a long context paired with a question requiring reasoning over the full passage. We treat each passage as a sequential stream of fixed-length (2048-token) segments. The objective is to consolidate information from the stream into model parameters, such that the model can answer the final query without access to the original context. To evaluate both consolidation and generalization, we consider two regimes: (1) **short**, with context lengths of 16k–32k tokens, and (2) **long**, with context lengths of 32k–64k tokens. The model is trained only on the short regime and evaluated on both. We evaluate on 40 short passages and 20 long passages. Due to the sparsity of the reward signal, evaluation is performed by sampling 10 candidate answers after consolidation and measuring mean accuracy. Additional details are provided in Section B.2.

Training procedure. We use Qwen2.5-7B-Instruct as our policy and reuse hyperparameters from Section 4. Meta-training consists of 2 rounds, each processing 5 passages (or streams). For each passage, at each step, we sample $K = 10$ candidate actions, perform inner-loop adaptation and use the intrinsic reward defined in Equation (17) to select the highest-scoring candidate. The selected adapter is merged into the running model (Equation (9)) before proceeding to the next segment.

Baselines. We compare methods across three settings: (1) **In-context. Base:** the base model answers questions without additional context. **Full Context:** the base model is provided the entire

passage. **Summarization**: the base model is given a concise summarization of the passage by Kimi-K2.6 [54]. **(2) Test-time training. Batch TTT**: a single LoRA adapter is trained on the full passage before querying. **Sequential FT**: Sequentially LoRA Fine-Tuning the model over passage segments without any module selection. **(3) Self-consolidation. SCoL**: our framework.

5.2 Results and discussions

Table 3 summarizes performance across both context regimes. SCoL achieves the strongest performance in the short-context setting at 42.3%, outperforming all baselines. In the long-context regime, SCoL remains competitive at 37.0%, significantly surpassing Sequential FT and exceeding in-context methods. These results demonstrate that explicit consolidation into model weights effectively integrates information across context lengths without requiring full access to the sequence. Batch TTT serves as a performance upper bound, achieving 41.5% in the long-context regime due to its global optimization over the entire sequence. In contrast, Sequential FT exhibits a performance collapse as context length increases (36.5% $\rightarrow$ 23.0%), underscoring the challenge of catastrophic forgetting under naïve online updates. SCoL consistently outperforms Sequential FT, confirming that structured consolidation is critical for stable continual adaptation. In the in-context setting, providing full context or a Kimi-K2.6 summary improves over the base model, but these gains diminish as context length increases. This degradation, consistent with prior findings that context length alone can hurt performance [55], emphasizes the limitations of relying solely on context or lossy summarization and motivates the transition to persistent parameter-based consolidation.

Overall, weight-consolidation approaches (SCoL and Batch TTT) systematically outperform in-context methods, highlighting the importance of updating model parameters rather than relying purely on transient context. Notably, although SCoL is meta-trained on short-context examples, it generalizes effectively to the long-context regime, handling sequences up to twice the training length while remaining competitive with methods that operate on full context. This suggests that our consolidation mechanism captures length-agnostic structure and can extrapolate beyond its training distribution. We hypothesize that further gains for SCoL could be achieved by scaling meta-training to longer sequences or dynamically adapting the strength of the forgetting term λ as a function of context length. We refer to section B.3 for additional results, discussions, and standard errors.

Table 3: Long-context consolidation comparison across two context-length regimes (short: 16k–32k and long: 32k–64k tokens).

Method	short	long
Base	32.5%	29.5%
Full Context	38.0%	35.5%
Summarization	39.3%	36.0%
Batch TTT	39.5%	41.5%
Sequential FT	36.5%	23.0%
SCoL	42.3%	37.0%

6 Conclusion, limitations, and future work

We introduced SCoL, a post-training framework that treats long-context inference as continual context consolidation. Instead of keeping new information only in the prompt or an external memory, SCoL post-trains the LLM to generate textual descriptions of where the current context should be written into its own weights. We formulate consolidation as meta-reinforcement learning problem, where each committed update changes the model state that will generate future update selections. Across SQuAD knowledge incorporation and LongBench v2 long-context consolidation, SCoL improves acquisition and retention over in-context, summarization, batch, and sequential test-time training baselines. Analysis of learned selection patterns shows that SCoL encourages sparse update locations that aligns with layers assigned high Fisher information, suggesting that the model learns to route plasticity toward loss-sensitive regions while limiting interference. Results on LongBench v2 further suggest that selection behavior learned on shorter streams can transfer to longer streaming.

Limitations. SCoL introduces computational overhead because rewards must be evaluated through inner adaptation. At each context, we sample candidate layer selections, adapt a copy of the current model for each candidate, evaluate acquisition and retention, and then commit the best update. Since each committed update changes the model state used for later contexts, this process is proceed sequentially along the stream, limiting parallelism across contexts. Additionally, SCoL could exhibit mode collapse over longer IPO updates, concentrating on repeated layer selections and reducing context-dependent diversity (see Appendix A.13). Future work should study policy update algorithms with improved diversity regularization, such as KL control or batch-level entropy penalties.

References

- [1] Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics*, 12:157–173, 2024.
- [2] Cheng-Ping Hsieh, Simeng Sun, Samuel Kriman, Shantanu Acharya, Dima Rekesh, Fei Jia, and Boris Ginsburg. RULER: What’s the real context size of your long-context language models? In *First Conference on Language Modeling*, 2024.
- [3] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems*, volume 33, 2020.
- [4] Sebastian Borgeaud, Arthur Mensch, Jordan Hoffmann, Trevor Cai, Eliza Rutherford, Katie Millican, George Bm van den Driessche, Jean-Baptiste Lespiau, Bogdan Damoc, Aidan Clark, Diego de Las Casas, Aurelia Guy, Jacob Menick, Roman Ring, Tom Hennigan, Saffron Huang, Loren Maggiore, Chris Jones, Albin Cassirer, Andy Brock, Michela Paganini, Geoffrey Irving, Oriol Vinyals, Simon Osindero, Karen Simonyan, Jack Rae, Erich Elsen, and Laurent Sifre. Improving language models by retrieving from trillions of tokens. In *Proceedings of the 39th International Conference on Machine Learning*, volume 162 of *Proceedings of Machine Learning Research*, pages 2206–2240. PMLR, 2022.
- [5] Weizhi Wang, Li Dong, Hao Cheng, Xiaodong Liu, Xifeng Yan, Jianfeng Gao, and Furu Wei. Augmenting language models with long-term memory. In *Advances in Neural Information Processing Systems*, volume 36, 2023.
- [6] Amirkeivan Mohtashami and Martin Jaggi. Random-access infinite context length for transformers. In *Advances in Neural Information Processing Systems*, volume 36, 2023.
- [7] Alexis Chevalier, Alexander Wettig, Anirudh Ajith, and Danqi Chen. Adapting language models to compress contexts. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 3829–3846, Singapore, 2023. Association for Computational Linguistics.
- [8] Huiqiang Jiang, Qianhui Wu, Chin-Yew Lin, Yuqing Yang, and Lili Qiu. LLMingua: Compressing prompts for accelerated inference of large language models. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 13358–13376, Singapore, 2023. Association for Computational Linguistics.
- [9] Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez. MemGPT: Towards LLMs as operating systems. *arXiv preprint arXiv:2310.08560*, 2023.
- [10] Wanjun Zhong, Lianghong Guo, Qiqi Gao, He Ye, and Yanlin Wang. Memorybank: Enhancing large language models with long-term memory. *Proceedings of the AAAI Conference on Artificial Intelligence*, 38(17):19724–19731, 2024.
- [11] James L. McClelland, Bruce L. McNaughton, and Randall C. O’Reilly. Why there are complementary learning systems in the hippocampus and neocortex: Insights from the successes and failures of connectionist models of learning and memory. *Psychological Review*, 102(3):419–457, 1995.
- [12] Dharshan Kumaran, Demis Hassabis, and James L. McClelland. What learning systems do intelligent agents need? complementary learning systems theory updated. *Trends in Cognitive Sciences*, 20(7):512–534, 2016.
- [13] Yadin Dudai, Avi Karni, and Jan Born. The consolidation and transformation of memory. *Neuron*, 88(1):20–32, 2015.
- [14] Michael McCloskey and Neal J. Cohen. Catastrophic interference in connectionist networks: The sequential learning problem. In *Psychology of Learning and Motivation*, volume 24, pages 109–165. Academic Press, 1989.

- [15] James Kirkpatrick, Razvan Pascanu, Neil Rabinowitz, Joel Veness, Guillaume Desjardins, Andrei A. Rusu, Kieran Milan, John Quan, Tiago Ramalho, Agnieszka Grabska-Barwinska, Demis Hassabis, Claudia Clopath, Dharshan Kumaran, and Raia Hadsell. Overcoming catastrophic forgetting in neural networks. *Proceedings of the National Academy of Sciences*, 114(13):3521–3526, 2017.
- [16] Gido M. van de Ven. On the computation of the fisher information in continual learning. *arXiv preprint arXiv:2502.11756*, 2025. To appear in the ICLR 2025 Blogpost Track.
- [17] Adam Zweiger, Jyothish Pari, Han Guo, Yoon Kim, and Pulkit Agrawal. Self-adapting language models. In *Advances in Neural Information Processing Systems*, 2025.
- [18] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations (ICLR)*, 2022.
- [19] Pranav Rajpurkar, Jian Zhang, Konstantin Lopyrev, and Percy Liang. SQuAD: 100,000+ questions for machine comprehension of text. In *Proceedings of the 2016 Conference on Empirical Methods in Natural Language Processing*, pages 2383–2392, Austin, Texas, 2016. Association for Computational Linguistics.
- [20] Yushi Bai, Shangqing Tu, Jiajie Zhang, Hao Peng, Xiaozhi Wang, Xin Lv, Shulin Cao, Jiazheng Xu, Lei Hou, Yuxiao Dong, Jie Tang, and Juanzi Li. LongBench v2: Towards deeper understanding and reasoning on realistic long-context multitasks. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 3639–3664, Vienna, Austria, 2025. Association for Computational Linguistics.
- [21] Kevin Meng, David Bau, Alex Andonian, and Yonatan Belinkov. Locating and editing factual associations in GPT. In *Advances in Neural Information Processing Systems*, volume 35, pages 17359–17372, 2022.
- [22] Eric Mitchell, Charles Lin, Antoine Bosselut, Chelsea Finn, and Christopher D. Manning. Fast model editing at scale. In *International Conference on Learning Representations (ICLR)*, 2022.
- [23] Peng Wang, Zexi Li, Ningyu Zhang, Ziwen Xu, Yunzhi Yao, Yong Jiang, Pengjun Xie, Fei Huang, and Huajun Chen. Wise: Rethinking the knowledge memory for lifelong model editing of large language models. *Advances in Neural Information Processing Systems*, 37:53764–53797, 2024.
- [24] Qizhou Chen, Taolin Zhang, Xiaofeng He, Dongyang Li, Chengyu Wang, Longtao Huang, and Hui Xue’. Lifelong knowledge editing for LLMs with retrieval-augmented continuous prompt learning. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pages 13565–13580, Miami, Florida, USA, 2024. Association for Computational Linguistics.
- [25] Suchin Gururangan, Ana Marasović, Swabha Swayamdipta, Kyle Lo, Iz Beltagy, Doug Downey, and Noah A. Smith. Don’t stop pretraining: Adapt language models to domains and tasks. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pages 8342–8360, Online, 2020. Association for Computational Linguistics.
- [26] Haizhou Shi, Zihao Xu, Hengyi Wang, Weiye Qin, Wenyuan Wang, Yibin Wang, Zifeng Wang, Sayna Ebrahimi, and Hao Wang. Continual learning of large language models: A comprehensive survey. *ACM Computing Surveys*, 58(5):1–42, 2025.
- [27] Hongyang Chen, Zhongwu Sun, Hongfei Ye, Kunchi Li, and Xuemin Lin. Continual learning in large language models: Methods, challenges, and opportunities. *arXiv preprint arXiv:2603.12658*, 2026.
- [28] Zhizhong Li and Derek Hoiem. Learning without forgetting. *IEEE transactions on pattern analysis and machine intelligence*, 40(12):2935–2947, 2018.

- [29] Xiao Wang, Tianze Chen, Qiming Ge, Han Xia, Rong Bao, Rui Zheng, Qi Zhang, Tao Gui, and Xuanjing Huang. Orthogonal subspace learning for language model continual learning. In *Findings of the Association for Computational Linguistics: EMNLP 2023*, pages 10658–10671, Singapore, 2023. Association for Computational Linguistics.
- [30] Renzhi Wang and Piji Li. LEMoE: Advanced mixture of experts adaptor for lifelong model editing of large language models. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pages 2551–2575, Miami, Florida, USA, 2024. Association for Computational Linguistics.
- [31] Kelvin Guu, Kenton Lee, Zora Tung, Panupong Pasupat, and Mingwei Chang. Retrieval augmented language model pre-training. In *Proceedings of the 37th International Conference on Machine Learning*, volume 119 of *Proceedings of Machine Learning Research*, pages 3929–3938. PMLR, 2020.
- [32] Jack W Rae, Anna Potapenko, Siddhant M Jayakumar, and Timothy P Lillicrap. Compressive transformers for long-range sequence modelling. *arXiv preprint arXiv:1911.05507*, 2019.
- [33] Joon Sung Park, Joseph C. O’Brien, Carrie J. Cai, Meredith Ringel Morris, Percy Liang, and Michael S. Bernstein. Generative agents: Interactive simulacra of human behavior. In *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology*, New York, NY, USA, 2023. Association for Computing Machinery.
- [34] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems*, volume 36, 2023.
- [35] Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Gray, John Schulman, Jacob Hilton, Fraser Kelton, Luke Miller, Maddie Simens, Amanda Askell, Peter Welinder, Paul Christiano, Jan Leike, and Ryan Lowe. Training language models to follow instructions with human feedback. In *Advances in Neural Information Processing Systems*, volume 35, 2022.
- [36] DeepSeek-AI. DeepSeek-R1 incentivizes reasoning in LLMs through reinforcement learning. *Nature*, 645:633–638, 2025.
- [37] Thomas Anthony, Zheng Tian, and David Barber. Thinking fast and slow with deep learning and tree search. In *Advances in Neural Information Processing Systems 30 (NIPS)*, 2017.
- [38] Qi Sun, Edoardo Cetin, and Yujin Tang. Transformer-squared: Self-adaptive llms. *arXiv preprint arXiv:2501.06252*, 2025.
- [39] Nathan Hu, Eric Mitchell, Christopher Manning, and Chelsea Finn. Meta-learning online adaptation of language models. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 4418–4432, Singapore, 2023. Association for Computational Linguistics.
- [40] Kevin Meng, Arnab Sen Sharma, Alex Andonian, Yonatan Belinkov, and David Bau. Mass-editing memory in a transformer. In *International Conference on Learning Representations (ICLR)*, 2023.
- [41] Song Wang, Yaochen Zhu, Haochen Liu, Zaiyi Zheng, Chen Chen, and Jundong Li. Knowledge editing for large language models: A survey. *arXiv preprint arXiv:2310.16218*, 2024.
- [42] Jeremy Howard and Sebastian Ruder. Universal language model fine-tuning for text classification. In *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (ACL)*, pages 328–339, 2018.
- [43] Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J. Liu. Exploring the limits of transfer learning with a unified text-to-text transformer. *Journal of Machine Learning Research*, 21(140):1–67, 2020.

- [44] Mohammad Gheshlaghi Azar, Mark Rowland, Bilal Piot, Daniel Guo, Daniele Calandriello, Michal Valko, and Rémi Munos. A general theoretical paradigm to understand learning from human preferences. In *Proceedings of the 27th International Conference on Artificial Intelligence and Statistics (AISTATS)*, 2024.
- [45] Caglar Gulcehre, Tom Le Paine, Srivatsan Srinivasan, Ksenia Konyushkova, Lotte Weerts, Abhishek Sharma, Aditya Siddhant, Alex Ahern, Miaosen Wang, Chenjie Gu, Wolfgang Macherey, Arnaud Doucet, Orhan Firat, and Nando de Freitas. Reinforced self-training (ReST) for language modeling. *arXiv preprint arXiv:2308.08998*, 2023.
- [46] Shouyuan Chen, Sherman Wong, Liangjian Chen, and Yuandong Tian. Extending context window of large language models via positional interpolation. *arXiv preprint arXiv:2306.15595*, 2023.
- [47] Jesse Mu, Xiang Lisa Li, and Noah Goodman. Learning to compress prompts with gist tokens. In *Advances in Neural Information Processing Systems 36 (NeurIPS)*, 2023.
- [48] Lukas Berglund, Meg Tong, Max Kaufmann, Mikita Balesni, Asa Cooper Stickland, Tomasz Korbak, and Owain Evans. The reversal curse: LLMs trained on “A is B” fail to learn “B is A”. In *International Conference on Learning Representations*, 2024.
- [49] Weijieying Ren, Xinlong Li, Lei Wang, Tianxiang Zhao, and Wei Qin. Analyzing and reducing catastrophic forgetting in parameter efficient tuning, 2024.
- [50] Mor Geva, Roei Schuster, Jonathan Berant, and Omer Levy. Transformer feed-forward layers are key-value memories. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, 2021.
- [51] Friedemann Zenke, Ben Poole, and Surya Ganguli. Continual learning through synaptic intelligence. In *Proceedings of the 34th International Conference on Machine Learning*, volume 70 of *Proceedings of Machine Learning Research*, pages 3987–3995. PMLR, 2017.
- [52] Arun Mallya and Svetlana Lazebnik. PackNet: Adding multiple tasks to a single network by iterative pruning. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 7765–7773, 2018.
- [53] Joan Serra, Dídac Surís, Marius Miron, and Alexandros Karatzoglou. Overcoming catastrophic forgetting with hard attention to the task. In *Proceedings of the 35th International Conference on Machine Learning*, volume 80 of *Proceedings of Machine Learning Research*, pages 4548–4557. PMLR, 2018.
- [54] Moonshot AI. Kimi k2.6 tech blog: Advancing open-source coding, 2026. Accessed: 2026-05-05.
- [55] Yufeng Du, Minyang Tian, Srikanth Ronanki, Subendhu Rongali, Sravan Bodapati, Aram Galstyan, Azton Wells, Roy Schwartz, Eliu A Huerta, and Hao Peng. Context length alone hurts llm performance despite perfect retrieval. *arXiv preprint arXiv:2510.05381*, 2025.
- [56] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *International Conference on Learning Representations*, 2019.
- [57] Rafael Rafailov, Archit Sharma, Eric Mitchell, Stefano Ermon, Christopher D. Manning, and Chelsea Finn. Direct preference optimization: Your language model is secretly a reward model. In *Advances in Neural Information Processing Systems 36 (NeurIPS)*, 2023.
- [58] Laura O’Mahony, Léo Grinsztajn, Hailey Schoelkopf, and Stella Biderman. Attributing mode collapse in the fine-tuning of large language models. In *ICLR 2024 Workshop on Mathematical and Empirical Understanding of Foundation Models*, 2024.
- [59] Ziniu Li, Congliang Chen, Tian Xu, Zeyu Qin, Jiancong Xiao, Zhi-Quan Luo, and Ruoyu Sun. Preserving diversity in supervised fine-tuning of large language models. *arXiv preprint arXiv:2408.16673*, 2024.

- [60] Jason Wei, Yi Tay, Rishi Bommasani, Colin Raffel, Barret Zoph, Sebastian Borgeaud, Dani Yogatama, Maarten Bosma, Denny Zhou, Donald Metzler, et al. Emergent abilities of large language models. *arXiv preprint arXiv:2206.07682*, 2022.
- [61] Jinze Bai, Shuai Bai, Yunfei Chu, Zeyu Cui, Kai Dang, Xiaodong Deng, Yang Fan, Wenbin Ge, Yu Han, Fei Huang, Binyuan Hui, Luo Ji, Mei Li, Junyang Lin, Runji Lin, Dayiheng Liu, Gao Liu, Chengqiang Lu, Keming Lu, Jianxin Ma, Rui Men, Xingzhang Ren, Xuancheng Ren, Chuanqi Tan, Sinan Tan, Jianhong Tu, Peng Wang, Shijie Wang, Wei Wang, Shengguang Wu, Benfeng Xu, Jin Xu, An Yang, Hao Yang, Jian Yang, Shusheng Yang, Yang Yao, Bowen Yu, Hongyi Yuan, Zheng Yuan, Jianwei Zhang, Xingxuan Zhang, Yichang Zhang, Zhenru Zhang, Chang Zhou, Jingren Zhou, Xiaohuan Zhou, and Tianhang Zhu. Qwen technical report. *arXiv preprint arXiv:2309.16609*, 2023.

A Continual knowledge injection: implementation details

This appendix collects the reproducibility details for the experiments of Section 4.

A.1 Selection prompt

The selection prompt is a Qwen chat-template instance shown in Figure 4. The prompt is parameterised by the passage title, the passage context, the implication string, the per-round budget B , and the maximum layer index. We instantiate $B = 10$ and the maximum layer index at 27 for Qwen2.5-7B.

```
<|im_start|>system
You are an assistant that selects which Transformer layers to update
so the model best memorizes the given passage and its implications.
<|im_end|>
<|im_start|>user
{title}
{context}

Implications:
{implications}

Select up to {budget} layer indices most critical for encoding this
knowledge. Each layer is an integer in  $[0, \{max\_layer\}]$ . Be selective.
Output ONLY comma-separated integers, no descriptions.<|im_end|>
<|im_start|>assistant
```

Figure 4: Selection prompt template.

The model’s response is parsed deterministically: all integers in the output are extracted by regex, filtered to the range $[0, L - 1]$, deduplicated preserving first-seen order, and truncated to the budget B . Each retained layer index is expanded to its seven projection matrices for adapter attachment. Malformed outputs that yield an empty parse produce an empty action, which in turn produces a near-zero reward under the inner adaptation of Equation (5); such rollouts are retained in the preference buffer but are, by construction, almost always the rejected element of any pair that includes them.

A.2 Implication generation

Implications are generated by the SEAL-pretrained Qwen checkpoint after its second self-training round, held frozen throughout all our experiments. The generation prompt is shown in Figure 5.

```
<|im_start|>system
You are an assistant tasked with analyzing the provided passage and
producing a list of implications derived directly or indirectly from
the content.
<|im_end|>
<|im_start|>user
{title}
{context}<|im_end|>
<|im_start|>assistant
```

Figure 5: Implication-generation prompt template, used unmodified from SEAL.

Sampling uses temperature 0.7, top- p 0.95, a maximum of 512 new tokens, and a single candidate per passage. The resulting implication string is cached per split and reused verbatim across all runs (ours and baselines), so implication-generation stochasticity does not contribute to variance between methods.

For inner adaptation, each cached implication string is split on -- and newlines, capped at 30 sequences, and each sequence is templated as $\{title\}\backslash n\{sequence\}$. The raw passage context

is appended as an additional sequence, so the final training set for one rollout contains at most 31 sequences. This templating is inherited from SEAL.

A.3 Inner LoRA configuration

Table 4 lists the inner adaptation hyperparameters.

Table 4: Inner LoRA configuration for one candidate rollout.

Field	Value
Rank r	32
Alpha α	64
Dropout	0.0
Optimiser	AdamW [56]
Learning rate	2×10^{-4}
Epochs	10
Batch size	1
Gradient accumulation	1
Warmup / scheduler	none

The commit policy merges the winning candidate’s adapter into the base weights at the end of each context and discards the others, so the running base model θ_t at step t is a single fully-consolidated Qwen2.5-7B checkpoint rather than a stack of adapters.

A.4 Outer IPO configuration

Table 5 lists the outer preference-optimisation configuration.

Table 5: Outer-loop IPO configuration.

Field	Value
Loss	IPO [44]
β	0.5
Preference-margin threshold	0.05 (pairs with $ r_w - r_l < 0.05$ dropped)
Pairs per context	all $\binom{K}{2} = 45$ surviving pairs
Optimiser	AdamW [56]
Learning rate	5×10^{-6}
Gradient accumulation	4
Epochs per round	2
Reference-policy snap	at start of each round; re-anchored after update

The reference policy π_{ref} for the round- r IPO update is the round-start base model $\pi_{\theta(r)}$ before any rollout-induced drift, matching the formulation in Section 3.2. After the IPO step, the updated base model becomes $\pi_{\theta(r+1)}$ and is used both as the new rollout substrate for round $r + 1$ and as the new reference policy for that round’s outer update.

A.5 Reward and judge

The QA judge is a Qwen2.5-7B-Instruct model served as a separate vLLM instance alongside the editable model. Each judgement receives a single (question, gold answer, student answer) triple through the prompt in Figure 6 and returns a binary decision.

The judge decodes greedily with a maximum of 8 new tokens. A regex of the form `\b(yes|no)\b` is run over the output and the last match is taken as the decision. Outputs whose last match is not “yes” (including outputs that produce no match at all) are scored zero. Accuracy on a single $\mathcal{Q}_t$ is the mean of the binary decisions over the roughly five questions in that passage.

You are a grading assistant. Your job is to determine whether a student’s answer correctly answers the question based solely on the provided gold answer. Do not use any outside knowledge. The student answer can include additional information, but it must at least fully convey the gold answer and must not contradict it. Ignore style, phrasing, or extra details that do not affect correctness. Respond ONLY with ‘yes’ or ‘no’.

Question: {question}
Gold answer: {gold}
Student answer: {pred}
Is the student answer correct based solely on the gold answer?
Respond ‘yes’ or ‘no’.

Figure 6: Judge prompt.

A.6 Metric formulas

All main-text metrics are summary statistics of the sequential-edit accuracy matrix $M \in [0, 1]^{N \times N}$, with $N = 100$ the length of the evaluation stream. Writing $M_{i,j}$ for the accuracy on $\mathcal{Q}_j$ after editing through c_i ($j \leq i$), we report

$$\begin{aligned} \text{Immediate acquisition} &= \frac{1}{N} \sum_i M_{i,i}, \\ \text{Retention} &= \frac{1}{N-1} \sum_{j < N-1} M_{N-1,j}, \end{aligned}$$

A.7 Stream and round configuration

Table 6 lists the continual-stream configuration.

Table 6: Stream configuration for Section 4.

Field	Value
Training passages per round T	50
Rounds reported in main text R	2 (plus round 0)
Rollouts per context K	10
Selector sampling temperature	1.0
Shuffle seed	42 (applied once before slicing)
Round-to-passages mapping	R1: passages [0, 50); R2: passages [50, 100)
Evaluation stream length	100 passages (fixed order)
Past-query subsampling	none

The training cache contains 250 shuffled passages; Section 4 uses the first two disjoint slices of 50 passages each. The remaining 150 passages are reserved for extended-rounds diagnostics (??) and are not used by any main-text result.

A.8 Baseline implementation details

QA Prompting. Prompting the base model with question only.

Batch LoRA FT. LoRA attached to all 28 layers, trained jointly on the concatenated implications of all 100 evaluation passages for 10 epochs at learning rate 2×10^{-4} .

SEAL. The SEAL knowledge-incorporation method applied to our continual stream, implemented as a sequential LoRA update per passage with the inner LoRA configuration of Table 4.

SCoL, both variants. Identical configurations; the only difference is the value of the reward weight λ .

A.9 Additional results

See standard errors in Table 7

Table 7: Standard errors for round-2 SQuAD sequential editing results over $K = 3$ inner-training seeds.

Method	Immediate acc.	Retention
SEAL	0.017	0.012
SCoL $_{\lambda=0}$	0.020	0.016
SCoL	0.008	0.013

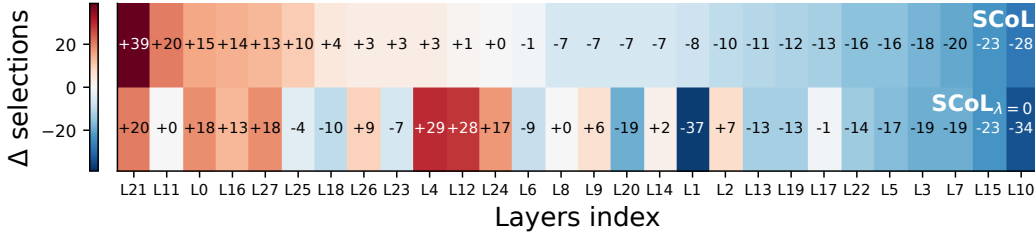


Figure 7: Per-layer change in selection count between the initial and final IPO updates. Layers are sorted by SCoL’s largest gain (left) to largest drop (right). SCoL (top, full reward) concentrates onto a handful of layers and abandons most others, while SCoL $_{\lambda=0}$ (bottom, acquisition only) shifts selection mass differently.

A.10 Compute resources

All experiments ran on NVIDIA RTX PRO 6000 Blackwell GPUs (96 GB each). A single training round ($K=10$ rollouts, $T=50$ passages) took approximately 4.3 hours per configuration. A single sequential matrix evaluation on the $N=100$ validation stream took 50 to 110 minutes on one GPU.

A.11 Outer Loop Algorithm Sweep: IPO, DPO, and ReST

We sweep three outer loop policy update algorithms while holding the inner loop, reward, stream, and rollout budget fixed. Each selector is trained on one round of $T=50$ passages with $K=10$ rollouts per passage, layer granularity, $\lambda=1$, and evaluated on the same $N=100$ validation stream. We compare ReST [45], DPO [57], and IPO from Eq. (8) [44]. For each algorithm, we sweep learning rate, gradient accumulation, and the algorithm specific parameters. All other hyperparameters match Table 2.

Following the mode concentration concern in Section 3.2, we report selector uniqueness as a diagnostic. For each evaluated selector, we count the number of *distinct* selection completions emitted across the 100 validation passages, denoted $\text{uniq}/100$. We also report $\text{top1\%}$, the share of the most common completion. $\text{uniq}/100$ is necessary but not sufficient for a useful selector, since high uniqueness can also reflect noisy selections. We therefore read it jointly with immediate accuracy.

IPO is the only update rule that jointly preserves $\text{uniq}/100$ above roughly 65 and reaches the strongest immediate accuracy, with 36.60% to 38.00% immediate accuracy and 65.80% to 69.30% retention at $\beta=0.5$. ReST shows the failure mode discussed in Section 3.2. Its aggressive setting reaches 41.00% immediate accuracy, but retention drops to 27.30%, while the most collapsed ReST setting has only 14 distinct completions and $\text{top1\%}$ of 61%. This trend is consistent with mode concentration under cross entropy style updates [58, 59]. DPO retains high $\text{uniq}/100$, often between 77 and 83, but underperforms IPO on the joint of immediate accuracy and retention. This sweep motivates IPO with $\beta=0.5$ as the headline outer loop algorithm, and we use this configuration for all main text results.

Table 8: Outer loop algorithm sweep. Retention uses the JSON summary key from `seq_eval.json`, not the paper retention metric in Section A.6.

Algorithm	Key hyperparameters	Imm. Acc.	Retention	uniq/100	top1 %
IPO	$\beta=0.5$	36.60%	69.30%	65	8
IPO	$\beta=0.5$, $ep=2$, $ga=4$, $lr=5\times 10^{-6}$	38.00%	65.80%	73	5
IPO	$\beta=0.1$	33.90%	54.30%	81	3
IPO	$\beta=0.01$	34.20%	74.40%	79	7
DPO	$\beta=0.1$	35.90%	42.20%	83	8
DPO	$\beta=0.5$	33.70%	66.30%	77	6
ReST	$lr=5\times 10^{-5}$, $ga=2$, $k=1$	35.90%	74.20%	59	9
ReST	$lr=3\times 10^{-4}$, $ga=10$, $k=1$	41.00%	27.30%	67	7
ReST	$lr=3\times 10^{-4}$, $ga=2$, $k=3$	12.90%	57.80%	14	61

Table 9: Round 2 granularity comparison. SCoL uses the full reward. SCoL $_{\lambda=0}$ removes the forgetting term from the reward.

Granularity	B	SCoL			SCoL $_{\lambda=0}$		
		Imm.	Ret.	uniq/100	Imm.	Ret.	uniq/100
Layer	10	35.40%	20.46%	82	34.30%	14.64%	85
Module	20	35.10%	20.21%	100	36.61%	19.45%	95
Per projection	70	24.70%	16.60%	99	31.40%	14.50%	96
Per projection	100	29.80%	20.00%	98	29.50%	17.00%	95

A.12 Granularity Comparison: Layer, Module, and Per Projection

The action space in Eq. (4) ranges over L structural units of the base model. We instantiate three choices in the same codebase, with budgets chosen to keep the per rollout adapter parameter count comparable. The layer setting uses $L=28$ and budget $B=10$, where each selected layer attaches LoRA adapters to seven projection modules. The module setting uses $L=56$ and budget $B=20$, where each layer is split into attention and MLP units. The per projection setting uses $L=196$ with budgets $B=70$ and $B=100$, where each q, k, v, o, gate, up, and down projection is a separate slot. All settings share the inner LoRA configuration in Table 1, the IPO configuration in Table 2, the same stream, $K=10$, and both reward variants.

Layer and module granularity yield comparable Round 2 immediate accuracy, around 35.00% across both reward variants, but the layer configuration is more stable across rounds and is therefore used in the main text. Per projection granularity underperforms by roughly five to ten absolute points in immediate accuracy, despite its larger nominal action space. Since `uniq/100` remains high, often above 90, the failure is better explained as selector *noise* rather than selector collapse.

This pattern suggests that action space expressivity is bottlenecked by structured generation. As granularity becomes finer, the policy must emit longer and more constrained outputs, moving from comma separated layer indices to module pairs and then projection pairs. With budgets of 70 or 100, the policy must produce a long list of valid pairs whose indices and projection names obey the parser grammar. A 7B base policy struggles with this output length and schema fidelity, so the effective entropy of useful rollouts shrinks even as the nominal action space grows.

Two future directions follow. First, the gap between layer and per projection granularity may narrow with stronger base policies, since long structured generation is plausibly scaling sensitive [60]. We state this only as a hypothesis. Second, the outer loop update can be made more diversity preserving at extreme structured lengths, either through entropy regularization [59] or through an objective that scores list *compositions* rather than independent positions. We leave both directions to future work.

A.13 Extended Round Dynamics and Mode Collapse

We also continue the outer loop beyond the two rounds reported in the main text. This experiment is diagnostic rather than a headline result. We run two additional rounds, R3 and R4, for layer and

module granularity under both reward variants. The inner loop, IPO objective, stream, and budget are unchanged. After each round, we record uniq/100 and immediate accuracy.

Table 10: Extended round diagnostics. Each cell reports immediate accuracy and uniq/100.

Granularity and reward	R1	R2	R3	R4
Layer, SCoL $_{\lambda=0}$	34.31% / 64	35.40% / 82	33.50% / 39	36.50% / 5
Layer, SCoL	35.16% / 85	34.30% / 85	37.50% / 69	39.10% / 33
Module, SCoL $_{\lambda=0}$	27.30% / 93	35.10% / 100	6.70% / 99	11.00% / 63
Module, SCoL	31.40% / 96	36.60% / 95	34.30% / 76	27.60% / 7

uniq/100 declines with additional rounds, and the full reward variant collapses faster than the acquisition only variant. On layer granularity, SCoL drops from 82 distinct completions at R2 to 5 at R4, while SCoL $_{\lambda=0}$ retains 33 at R4. The module setting shows the same direction, with the full reward variant also collapsing in immediate accuracy to 6.70% at R3.

Round 2 should therefore be read as an early stopping point for selector quality, not as the asymptotic behavior of the fixed objective. With the same IPO loss and buffer construction, later rounds concentrate probability mass on a small number of high reward actions observed during training and reduce exploration. This is the mode concentration mechanism flagged in Section 3.2 and observed in cross entropy and preference style fine tuning of LLMs [58, 59]. For this reason, the main text reports R0 to R2 and treats further round scaling as a regularization problem for future work.

A.14 Layerwise Fisher computation

For each passage c , let $\mathcal{I}(c)$ denote its generated implication lines. Fisher scores are computed on the running consolidated base model before attaching a new LoRA adapter for the passage. For each implication line $x \in \mathcal{I}(c)$, we compute the autoregressive sum loss

$$\mathcal{L}(x; \theta) = - \sum_{t=1}^{|x|} \log p_{\theta}(x_t \mid x_{<t}),$$

where θ denotes the current consolidated base parameters. We estimate diagonal Fisher importance by squared gradients:

$$F_j(c) = \frac{1}{|\mathcal{I}(c)|} \sum_{x \in \mathcal{I}(c)} \left(\frac{\partial \mathcal{L}(x; \theta)}{\partial \theta_j} \right)^2.$$

Let $\mathcal{P}_{\ell}$ be the parameters of the seven base projection matrices in layer ℓ that are eligible for LoRA attachment. The layerwise Fisher score is

$$F_{\ell}(c) = \sum_{j \in \mathcal{P}_{\ell}} F_j(c), \quad \ell \in \{0, \dots, 27\}.$$

This yields a 28 dimensional Fisher vector $F(c)$ for each passage. These scores are used only for analysis and are never provided to the LLM when it generates layer selections.

B Long-context Consolidation

B.1 Hyperparameters

B.1.1 Inner LoRA configuration

We retain the same LoRA configuration from Section A.9 and Table 4. Additionally, all our chunks are of a fixed 2048 token length calculated using the Qwen2.5-7B tokenizer and we create a training set by further chunking these into 128 token size subchunks.

B.1.2 Outer IPO configuration

We retain mostly the same IPO configuration from Section A.9 and Table 5. We only change the preference margin threshold from 0.05 to 0.3. That is, given candidates c_1, c_2 with rewards r_1, r_2 respectively and $r_1 > r_2$, we put $\{c_1, c_2\}$ in the IPO buffer iff $r_1 - r_2 > 0.2$.

B.2 Implementation Details

SCoL. We retain the majority of the setup from Section A, with the modifications presented in Section B.1

In-context settings. We use the base Qwen2.5-7B model with temperature = 1 implemented using vLLM.

For **Summarization**, we prompt Kimi-K2.6 using the template shown in Figure 8. We set maxTokens to 10,000 to discourage degenerate behavior where the model copies large portions of the original passage instead of compressing it.

```
Please provide a dense and concise summary of the following passage in under
{maxTokens} tokens keeping all the important information:\n\n{context}\n\nSummary:
```

Figure 8: Summarization prompt used for Kimi-K2.6.

Test-time training. For both Batch TTT and Sequential FT, we attach LoRA adapters to all 28 transformer layers. Training is performed either jointly on the full passage (Batch TTT) or incrementally over sequential chunks (Sequential FT). Each passage is evaluated using freshly initialized LoRA adapters trained under the same inner-loop test-time training configuration as given in Section B.1.

B.2.1 Dataset Processing

Our goal is to evaluate whether long contexts can be continually consolidated into model parameters. We therefore use LongBench-v2, which contains naturally long documents (e.g., books and multi-section texts) paired with reasoning-based question-answer tasks.

Each document is treated as an independent context and processed as a stream of 2048-token segments, simulating a setting where information arrives sequentially. For training, we use 10 passages with lengths between 16k and 32k tokens. We sample 40 different passages for evaluation and an additional 20 passages of lengths between 32k and 64k tokens.

Tokenization and segmentation are performed using the Qwen2.5-7B tokenizer. All dataset splits are created with a fixed random seed of 42 for reproducibility.

B.2.2 Compute Resources

All experiments for this section are conducted on NVIDIA H200 GPUs with 8 CPUs. Meta-training (2 rounds over 5 passages each) takes approximately 8 hours. Evaluation over the full benchmark (40 short and 20 long passages) requires approximately 6 hours.

B.3 Additional Results

Passage Log-likelihoods. We further compare the final Log-Likelihoods between batch TTT, sequential FT, and SCoL in Table 11. Consistent with the main results, Batch TTT achieves the highest likelihood in both regimes (-1.26), indicating that full-sequence optimization yields the best fit to the data distribution. In contrast, Sequential FT performs substantially worse (-2.32 short, -2.51 long), reinforcing that naïve online updates lead to degraded representations and poor predictive calibration.

SCoL consistently improves over Sequential FT in both regimes (-1.44 vs. -2.32 in short, -1.82 vs. -2.51 in long), indicating that structured consolidation mitigates the degradation in likelihood associated with continual updates. Notably, the gap between SCoL and Batch TTT is smaller in the short-context regime (0.18) than in the long-context regime (0.56), mirroring the trends observed in accuracy.

We hypothesize that this widening gap reflects the same stability-plasticity tradeoff observed earlier. In the short-context setting, SCoL’s forgetting-aware objective enables updates that

Table 11: Long-context consolidation comparison of final log-likelihoods across short and long regimes.

Method	short ($\uparrow$)	long ($\uparrow$)
Batch TTT	-1.26	-1.26
Sequential FT	-2.32	-2.51
SCoL	-1.44	-1.82

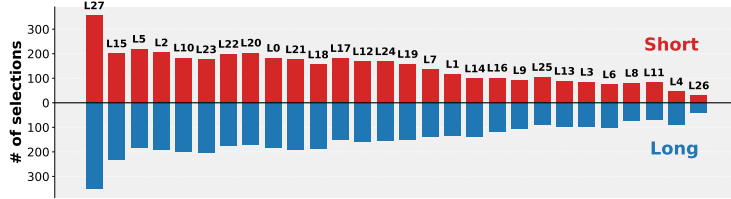


Figure 9: Layer selections for SCoL for the long context setting for short and long regimes.

remain close to the optimal batch solution, yielding likelihoods comparable to Batch TTT. However, as context length increases, the constraint imposed by the forgetting term may limit the model’s ability to fully adapt to the extended sequence, leading to lower likelihood relative to Batch TTT.

Overall, these results support the conclusion that SCoL provides a strong approximation to batch retraining while maintaining stability, substantially improving over naïve sequential updates, though some loss in optimality remains when full-context adaptation is required.

Layer selection frequencies. We also show the top layer selection frequencies for our Long Context Consolidation Setting in Figure 9. Similar to the results in Section 4, we find that L27 is the selected most often but here we find that L26 and L25 are selected relatively less. Additionally, the middle layers are selected least often matching the SQuAD setting.

Table 12: Long-context consolidation comparison across two context-length regimes (short: 16k–32k and long: 32k–64k tokens).

Method	short	long
Base	$32.5 \pm 0.4\%$	$29.5 \pm 0.5\%$
Full Context	$38.0 \pm 0.5\%$	$35.5 \pm 0.8\%$
Summarization	$39.3 \pm 0.5\%$	$36.0 \pm 0.7\%$
Batch TTT	$39.5 \pm 0.5\%$	$41.5 \pm 0.5\%$
Sequential FT	$36.5 \pm 0.8\%$	$23.0 \pm 1.0\%$
SCoL	$42.3 \pm 0.8\%$	$37.0 \pm 1.0\%$

Full results table with standard errors. We provide the full results table for the long context experiments, along with the standard errors in table Table 12.

C Broader Impacts

The proposed SCoL framework enables long-context internalization through persistent weight updates, offering significant improvements in computational efficiency and long-term agent memory. However, this shift from transient context to persistent weights introduces privacy risks, as consolidated sensitive information may be harder to redact. Furthermore, continual self-adaptation may inadvertently reinforce biases or internalize misinformation present in the input stream.

D Licenses

We provide details regarding the licenses of external assets used in this work, presented in Table 13.

Asset	URL	License
Datasets		
SQuAD [19]	Link	Creative Commons Attribution-ShareAlike 4.0 International
LongBench-v2 [20]	Link	MIT License
Models		
Qwen Models [61]	Link	Apache License 2.0*

Table 13: Licenses for assets used in our experiments. *The majority of the Qwen series (e.g., Qwen1.5, Qwen2, Qwen2.5) are licensed under Apache 2.0, with a few exceptions for specific legacy sizes governed by the Tongyi Qianwen License Agreement.